

Week 5

**Implement deep learning Techniques for image segmentation -
CNN – U-Net Model**

Aim of the Program

- ❖ To implement an image segmentation pipeline using a U-Net model in TensorFlow, which processes an input image to generate and visualize a binary segmentation mask by applying deep learning techniques.

1. Library Imports

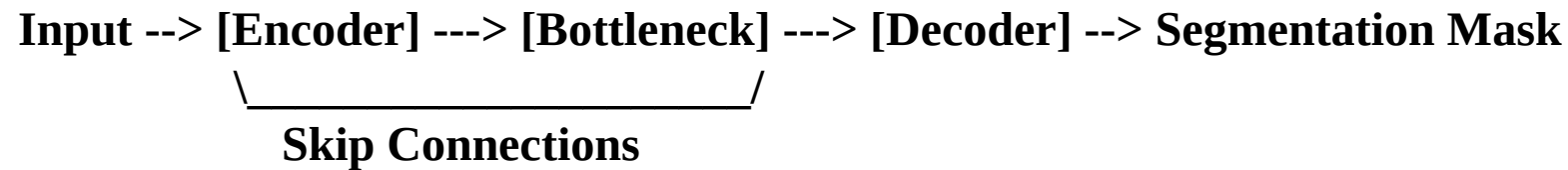
- ❖ **cv2OpenCV:** This library is used for image processing tasks, such as reading, resizing, and displaying images.
- ❖ **numpy:** It handles numerical operations on arrays (images are treated as arrays of pixel values).
- ❖ **tensorflow/keras:** These libraries are used for building and running machine learning models. Specifically, Keras simplifies defining and training neural networks.

2. Image Loading and Preprocessing

- ❖ **Image loading:** The image is loaded from a specified path using `cv2.imread()`. It's stored as an array where each pixel is represented by its RGB values.
- ❖ **Resizing:** Since neural networks typically require fixed input sizes, the image is resized to 128x128 pixels to fit the input layer of the U-Net model.
- ❖ **Normalization:** Neural networks work more efficiently when input values are in a small range. Dividing pixel values by 255.0 scales them between 0 and 1, making the model training more stable.

3. U-Net Model Architecture:

- ❖ The U-Net is a Convolutional Neural Network (CNN) architecture specifically designed for image segmentation.
- ❖ Overall Architecture: "U" Shape
- ❖ The U-Net has a symmetric "U"-shaped architecture with two main parts:
 - ❖ **Encoder (Contracting Path)** – left side
 - ❖ **Decoder (Expanding Path)** – right side
 - ❖ **Skip Connections** – connect matching layers from encoder to decoder



- ❖ An encoder is a key component responsible for feature extraction through a process of downsampling. It's the first half of a typical encoder-decoder architecture, which is a powerful framework used in tasks like image segmentation, image-to-image translation, and autoencoders (when working with high-dimensional data such as images, music, or text)
- ❖ Each block includes:
 - ❖ Two Conv2D layers with ReLU activation
 - ❖ One MaxPooling2D layer for downsampling
- ❖ **Bottleneck – Bridge Between Encoder and Decoder in U-Net**
 - ❖ The bottleneck is the central part of the U-Net architecture, sitting between the encoder and the decoder.
 - ❖ It captures abstract and complex features:
 - ❖ Texture, Shape, Global context
 - ❖ Think of it like the core "understanding" of the image

- ❖ **Decoder (Upsampling):** In the decoder, the image size is gradually increased using upsampling techniques (transposed convolutions). The goal is to reconstruct the image and predict which pixels belong to certain regions.
- ❖ **Skip connections:** A key feature of U-Net is the use of skip connections between the encoder and decoder. This allows the decoder to use high-resolution features from earlier layers in the encoder, which helps retain important details when reconstructing the image.

4. Model Compilation:

- ❖ **Adam optimizer:** This is an optimization algorithm used to adjust the model weights during training. It's popular for its adaptive learning rates, making training efficient.
- ❖ **Binary cross-entropy loss:** Since this is a binary segmentation task (where each pixel is classified as either belonging to the foreground or background), binary cross-entropy is the loss function that measures the difference between predicted and actual segmentation masks.
- ❖ **Accuracy:** The model's performance is evaluated using accuracy, which checks how often the predictions match the true segmentation mask.

5. Prediction

- ❖ **Preprocessing for prediction:** Since the model expects batches of images, the input image is reshaped to add a batch dimension (even if there's only one image).
- ❖ **Prediction:** The model outputs a segmentation map, which is essentially a probability score for each pixel indicating whether it belongs to the segmented region.

6. Post-processing

- ❖ **Binarization:** The predicted probabilities (between 0 and 1) are thresholded, i.e., values greater than 0.5 are set to 1 (indicating the pixel is part of the object), and values less than 0.5 are set to 0 (indicating the background).
- ❖ **Squeezing dimensions:** The extra batch dimension is removed, resulting in a binary segmentation mask.

7. Displaying and Saving Images

- ❖ **Displaying:** The original image and the segmented output are displayed using OpenCV.
- ❖ **Saving:** The binary segmentation mask is saved to a file, where pixel values are scaled to the range 0-255 (common for image formats).

8. Closing OpenCV Windows

- ❖ The program waits for a user keypress before closing the image windows, ensuring the user has time to view the results. After that, the OpenCV windows are closed to clean up resources.

1. pip install tensorflow opencv-python matplotlib

1. import cv2

2. import numpy as np

3. import matplotlib.pyplot as plt

4. import tensorflow as tf

5. from tensorflow.keras import layers

6. # Load the image

7. image_path = 'C:/Exp/1234.png'

8. image = cv2.imread(image_path)

9. image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

10. # Resize image to the desired input size of the model

11. target_size = (128, 128) # Change to your model's input size

12. image_resized = cv2.resize(image_rgb, target_size)

Normalize the image

1. image_normalized = image_resized / 255.0

Add batch dimension

1. image_batch = np.expand_dims(image_normalized, axis=0)

UNet Model Definition (You can also use a pre-trained model)

```
1. def build_unet_model(input_size=(128, 128, 3)):  
2.     inputs = layers.Input(input_size)  
3.     # Encoder  
4.     c1 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(inputs)  
5.     c1 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(c1)  
6.     p1 = layers.MaxPooling2D((2, 2))(c1)  
7.     c2 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(p1)  
8.     c2 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(c2)  
9.     p2 = layers.MaxPooling2D((2, 2))(c2)
```

```
1. c3 = layers.Conv2D(256, (3, 3), activation='relu', padding='same')(p2)
2. c3 = layers.Conv2D(256, (3, 3), activation='relu', padding='same')(c3)
3. p3 = layers.MaxPooling2D((2, 2))(c3)
4.
5. # Bottleneck
6. c4 = layers.Conv2D(512, (3, 3), activation='relu', padding='same')(p3)
7. c4 = layers.Conv2D(512, (3, 3), activation='relu', padding='same')(c4)
8.
9. # Decoder
10. u5 = layers.Conv2DTranspose(256, (2, 2), strides=(2, 2), padding='same')(c4)
11. u5 = layers.concatenate([u5, c3])
12. c5 = layers.Conv2D(256, (3, 3), activation='relu', padding='same')(u5)
13. c5 = layers.Conv2D(256, (3, 3), activation='relu', padding='same')(c5)
14.
15. u6 = layers.Conv2DTranspose(128, (2, 2), strides=(2, 2), padding='same')(c5)
16. u6 = layers.concatenate([u6, c2])
17. c6 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(u6)
18. c6 = layers.Conv2D(128, (3, 3), activation='relu', padding='same')(c6)
19.
```

1. `u7 = layers.Conv2DTranspose(64, (2, 2), strides=(2, 2), padding='same')(c6)`
2. `u7 = layers.concatenate([u7, c1])`
3. `c7 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(u7)`
4. `c7 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(c7)`
5. `outputs = layers.Conv2D(1, (1, 1), activation='sigmoid')(c7)`
6. `model = tf.keras.Model(inputs=[inputs], outputs=[outputs])`
7. `return model`
8. `model = build_unet_model()`
9. `model.summary()`

Model: "model"

Layer (type)	Output Shape	Param #	Connected to
=====			
input_1 (InputLayer)	[(None, 128, 128, 3)]	0	[]
conv2d (Conv2D)	(None, 128, 128, 64)	1792	['input_1[0][0]']
conv2d_1 (Conv2D)	(None, 128, 128, 64)	36928	['conv2d[0][0]']
max_pooling2d (MaxPooling2D)	(None, 64, 64, 64)	0	['conv2d_1[0][0]']
conv2d_2 (Conv2D)	(None, 64, 64, 128)	73856	['max_pooling2d[0][0]']
conv2d_3 (Conv2D)	(None, 64, 64, 128)	147584	['conv2d_2[0][0]']
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 128)	0	['conv2d_3[0][0]']
conv2d_4 (Conv2D)	(None, 32, 32, 256)	295168	['max_pooling2d_1[0][0]']
conv2d_5 (Conv2D)	(None, 32, 32, 256)	590080	['conv2d_4[0][0]']

conv2d_6 (Conv2D)	(None, 16, 16, 512)	1180160	['max_pooling2d_2[0][0]']
conv2d_7 (Conv2D)	(None, 16, 16, 512)	2359808	['conv2d_6[0][0]']
conv2d_transpose (Conv2DTranspose)	(None, 32, 32, 256)	524544	['conv2d_7[0][0]']
concatenate (Concatenate)	(None, 32, 32, 512)	0	['conv2d_transpose[0][0]', 'conv2d_5[0][0]']
conv2d_8 (Conv2D)	(None, 32, 32, 256)	1179904	['concatenate[0][0]']
conv2d_9 (Conv2D)	(None, 32, 32, 256)	590080	['conv2d_8[0][0]']
conv2d_transpose_1 (Conv2DTranspose)	(None, 64, 64, 128)	131200	['conv2d_9[0][0]']
concatenate_1 (Concatenate)	(None, 64, 64, 256)	0	['conv2d_transpose_1[0][0]', 'conv2d_3[0][0]']
conv2d_10 (Conv2D)	(None, 64, 64, 128)	295040	['concatenate_1[0][0]']
conv2d_11 (Conv2D)	(None, 64, 64, 128)	147584	['conv2d_10[0][0]']

conv2d_transpose_2 (Conv2D Transpose)	(None, 128, 128, 64)	32832	['conv2d_11[0][0]']
concatenate_2 (Concatenate)	(None, 128, 128, 128)	0	['conv2d_transpose_2[0][0]', 'conv2d_1[0][0]']
conv2d_12 (Conv2D)	(None, 128, 128, 64)	73792	['concatenate_2[0][0]']
conv2d_13 (Conv2D)	(None, 128, 128, 64)	36928	['conv2d_12[0][0]']
conv2d_14 (Conv2D)	(None, 128, 128, 1)	65	['conv2d_13[0][0]']

=====

Total params: 7697345 (29.36 MB)
 Trainable params: 7697345 (29.36 MB)
 Non-trainable params: 0 (0.00 Byte)

Compile the model

**1. `model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])`**

Example: `model.fit(x_train, y_train, epochs=10, batch_size=2, validation_split=0.1)`

For demonstration, we will assume the model is already trained.

1. **# Make a prediction**
2. **prediction = model.predict(image_batch)**
3. **# Post-process the prediction**
4. **predicted_mask = np.squeeze(prediction) # Remove batch dimension**
5. **predicted_mask = np.round(predicted_mask) # Binarize the mask**
6. **# Display the result**
7. **plt.figure(figsize=(12, 6))**
8. **plt.subplot(1, 3, 1)**
9. **plt.title('Original Image')**
10. **plt.imshow(image_rgb)**
11. **plt.axis('off')**
12. **plt.subplot(1, 3, 2)**
13. **plt.title('Resized Image')**
14. **plt.imshow(image_resized)**
15. **plt.axis('off')**
16. **plt.subplot(1, 3, 3)**
17. **plt.title('Predicted Mask')**
18. **plt.imshow(predicted_mask, cmap='gray')**
19. **plt.axis('off')**
20. **plt.show()**

1/1 [=====] - 3s 3s/step

Original Image



Resized Image



Predicted Mask



Summary:

The code implements an image segmentation workflow using a UNet model in TensorFlow. It loads an input image, resizes it to 128x128 pixels, and normalizes the pixel values. A UNet architecture is defined, featuring an encoder for downsampling and a decoder for upsampling to create segmentation maps. The model is compiled with the Adam optimizer and binary cross-entropy loss. After preprocessing the image for prediction, the model generates a segmentation output, which is thresholded into a binary mask. Finally, the original image and the segmented result are displayed, and the segmented image is saved to disk.